



INTERNATIONAL CYBERSECURITY & DIGITAL FORENSIC ACADEMY
(ICDFA)

INT301 Week 2 Lab

Operating Systems Fundamentals

Ethical Hacking and Digital Forensics Internship

Phase 1 - Week 2

Name: Olatunji Ramon Balogun

Student ID: 2025/INT/12584

INSTRUCTOR NAME: Mr. Aminu Idris AMCPA

2025/INT/12584

20 December 2025

Executive Summary

This lab scenario represented a similar environment that both system administrators and software developers use to automate repetitive tasks, facilitate the validation of input data, and monitor the performance of systems.

The lab for this course provided students with the opportunity to develop and demonstrate skills related to shell scripting. Students developed four required scripts: user input, password check, countdown, and sum function. In addition to these scripts, students selected two practice assignments they felt would further enhance their understanding of scripting in Shell. All assignments included an emphasis on input/output handling, using conditional logic for data validation, creating and using iterative loops, developing functions and reusing them to streamline code, and using system-based tracking tools (e.g., counting files, checking websites, etc.) to create and maintain their own environments.

The students' submissions provided a way for them to showcase their ability to design, create, execute, and document their scripts in a structured fashion. The use of a standard PDF submission format reinforced the importance of creating reusable code that is readable and maintainable. The completion of this lab provided each student with an opportunity to practice and enhance the skills needed to begin developing more complex operating systems

Introduction

Operating systems use automation and scripting to handle process management, security enforcement/examples of preventable range of mistakes made by a system administrator; for example, shell scripting is an important skill in the System Administrators or Programmers environment to allow them to interact directly with their OS environments as well as enabling them to build custom workflows.

This particular lab session focus on a combination of the student's ability to create, test and document a Bash script through various real-life case studies from input/output handling through basic functionality and system checking features and procedures. In addition, it provided an opportunity for the students to practice and demonstrate the level of professionalism when documenting their work by requiring them to include a combination of both program code along with several screen shots in one consolidated PDF document.

The goals of these labs were to:

- Build a solid foundation in Bash scripting syntax and execution basics.
- Provide hands-on experience in user input handling and implementing conditional statements in scripts.
- Provide students with an opportunity to use loops and functions to create automated scripts for repetitive tasks.
- Reinforce the importance of academic integrity by requiring that all scripts be original and properly documented.
- Provide students the opportunity to develop professional coding/reporting skills by developing a structured PDF report that includes screenshots of the script code, a description of how it works, and its output.
- Encourage students to investigate optional practice/bonus exercises as a means to learn additional concepts not covered in the main lab tasks

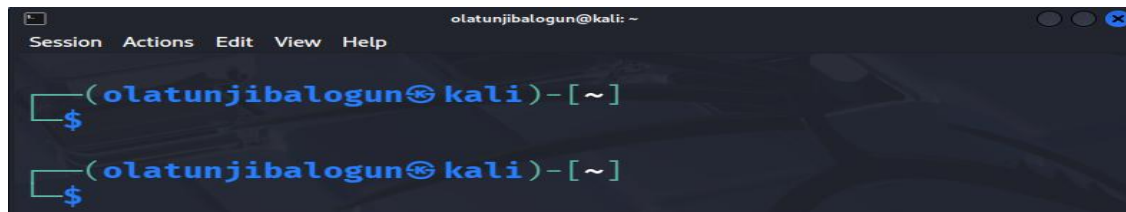
Part 1: Basic Scripting Exercises

Exercise 2.1: User Input Script

Task: Create a script that asks for your favorite color and food, then prints a sentence.

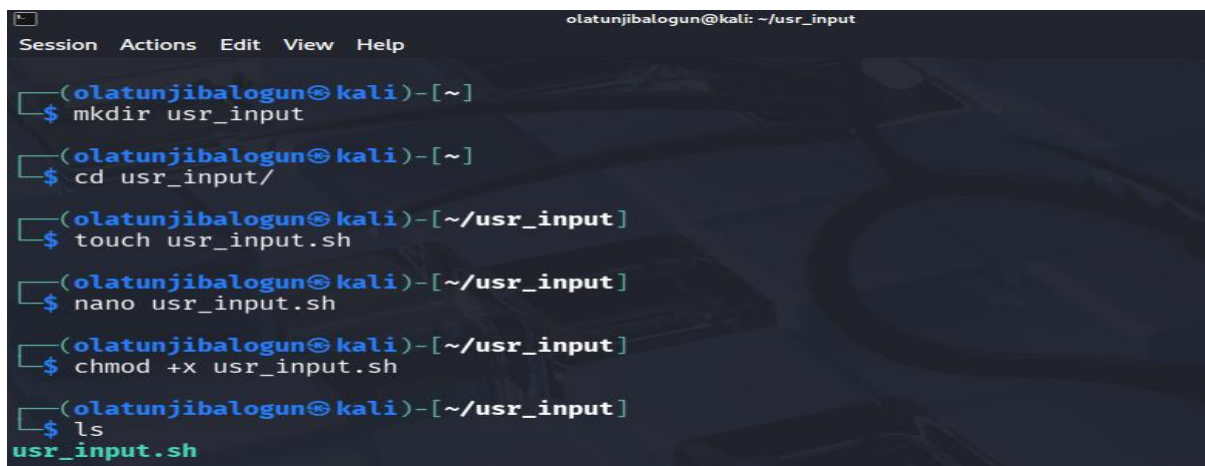
A screenshot showing:

- **Terminal with your username visible**

A screenshot of a terminal window with a dark background. The title bar at the top reads "olatunjibalogun@kali: ~". Below the title bar is a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal shows two prompts: "(olatunjibalogun@kali)-[~]" followed by a "\$" prompt, and then another "(olatunjibalogun@kali)-[~]" followed by a "\$" prompt.

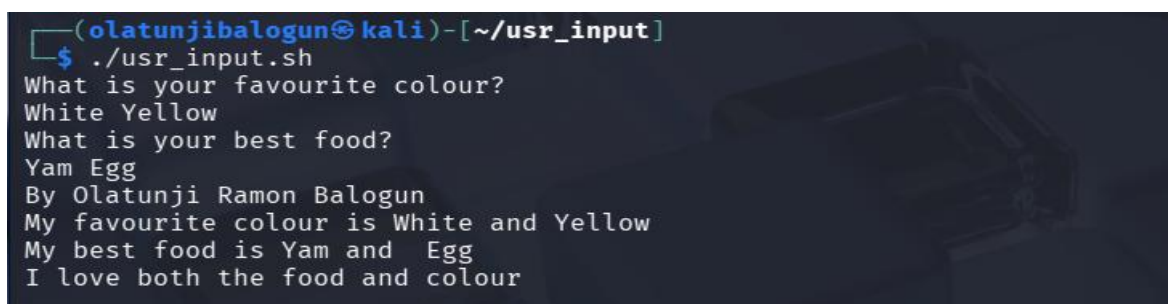
The terminal window displays the username **olatunjibalogun@kali**, showing that I ran my script from my own Linux system.

- **Script execution with your inputs**

A screenshot of a terminal window with a dark background. The title bar at the top reads "olatunjibalogun@kali: ~/usr_input". Below the title bar is a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal shows the following commands and their outputs: 1. Prompt: "(olatunjibalogun@kali)-[~]", Command: "\$ mkdir usr_input", Output: "(olatunjibalogun@kali)-[~]". 2. Prompt: "(olatunjibalogun@kali)-[~]", Command: "\$ cd usr_input/", Output: "(olatunjibalogun@kali)-[~/usr_input]". 3. Prompt: "(olatunjibalogun@kali)-[~/usr_input]", Command: "\$ touch usr_input.sh", Output: "(olatunjibalogun@kali)-[~/usr_input]". 4. Prompt: "(olatunjibalogun@kali)-[~/usr_input]", Command: "\$ nano usr_input.sh", Output: "(olatunjibalogun@kali)-[~/usr_input]". 5. Prompt: "(olatunjibalogun@kali)-[~/usr_input]", Command: "\$ chmod +x usr_input.sh", Output: "(olatunjibalogun@kali)-[~/usr_input]". 6. Prompt: "(olatunjibalogun@kali)-[~/usr_input]", Command: "\$ ls", Output: "usr_input.sh".

I created a folder with the command **mkdir usr_input**, then changed to it using **cd usr_input**. Then I created a shell script file called **usr_input.sh** using **touch usr_input.sh**, edited it using **nano usr_input.sh**, made it executable using **chmod +x usr_input.sh** and finally listed it using **ls**.

- **Output showing the combined sentence**

A screenshot of a terminal window with a dark background. The title bar at the top reads "olatunjibalogun@kali: ~/usr_input". Below the title bar is a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal shows the following commands and their outputs: 1. Prompt: "(olatunjibalogun@kali)-[~/usr_input]", Command: "\$./usr_input.sh", Output: "What is your favourite colour?". 2. Input: "White Yellow", Output: "What is your best food?". 3. Input: "Yam Egg", Output: "By Olatunji Ramon Balogun". 4. Output: "My favourite colour is White and Yellow". 5. Output: "My best food is Yam and Egg". 6. Output: "I love both the food and colour".

The script was invoked using **./usr_input.sh**, then I entered my favorite colors (**'White' and 'Yellow'**) and favorite foods (**'Yams' and 'Eggs'**). Following those entries made, the script displayed a statement built using those entries which stated a preference.

```

(olatumjibalogun@kali)-[~/usr_input]
$ cat usr_input.sh
#!/bin/bash

#asking about the colour?
echo "What is your favourite colour?"
read colour colour2 #user input

#asking about the food?
echo "What is your best food?"
read food food2 #user input

#The sentence
echo "By Olatunji Ramon Balogun"
echo "My favourite colour is $colour and $colour2" #output
echo "My best food is $food and $food2"
echo "I love both the food and colour"

```

This shows the complete listing of my **sudo script** **usr_input**. The script contains prompts for input as well as assignments to variables. The script utilizes read commands to capture the 2 color choices and food selections and then displays an output statement combining both along with my name using echo.

```

#!/bin/bash

#asking about the colour?
echo "What is your favourite colour?"
read colour colour2 #user input

#asking about the food?
echo "What is your best food?"
read food food2 #user input

#The sentence
echo "By Olatunji Ramon Balogun"
echo "My favourite colour is $colour and $colour2" #output
echo "My best food is $food and $food2"
echo "I love both the food and colour"

```

Exercise 3.1: Password Checker

Task: Create a password checker for "secret123".

Script code

```

(olatumjibalogun@kali)-[~]
$ cd passchecker

(olatumjibalogun@kali)-[~/passchecker]
$ touch passchecker.sh

(olatumjibalogun@kali)-[~/passchecker]
$ nano passchecker.sh

(olatumjibalogun@kali)-[~/passchecker]
$ chmod +x passchecker.sh

```

```
(olatumjibalogun@kali)-[~/passchecker]
$ cat passchecker.sh
#!/bin/bash

#Enter your password
echo "Enter Password:"
read password                #input Password

if [ $password = Secret123 ]; then #password
echo "Successfull Access"         #Secret123 Successful
else
echo "Wrong Access"              #Wrong
fi
```

The screenshot below shows the complete contents of the password-checking script named `passchecker.sh`. The script prompts for the user's password, checks that password against the expected value of `Secret123`, and will output either "Successful Access" if the entered password matches, or "Wrong Access" if it does not.

- Screenshot 1: Successful access with "secret123."

```
(olatumjibalogun@kali)-[~/passchecker]
$ ./passchecker.sh
Enter Password:
Secret123
Successfull Access
```

In this screenshot, the example execution of the script shows that when the correct password of `Secret123` was entered, the output message verifies successful access and that the conditional logic in the script performed as expected.

- Screenshot 2: Failed access with the wrong password

```
(olatumjibalogun@kali)-[~/passchecker]
$ ./passchecker.sh
Enter Password:
Possibility@34567
Wrong Access
```

This screenshot is an example execution of the script with an incorrect password of `Possibility@34567`. In this case, the output message is "Wrong Access," indicating that the script properly rejected an invalid password as expected.

```
#!/bin/bash

#Enter your password
echo "Enter Password:"
read password                #input Password

if [ $password = Secret123 ]; then #password
echo "Successfull Access"         #Secret123 Successful
else
echo "Wrong Access"              #Wrong
fi
```

Exercise 4.1: Countdown Script

Task: Count from 10 to 1, then print "Blast off!".

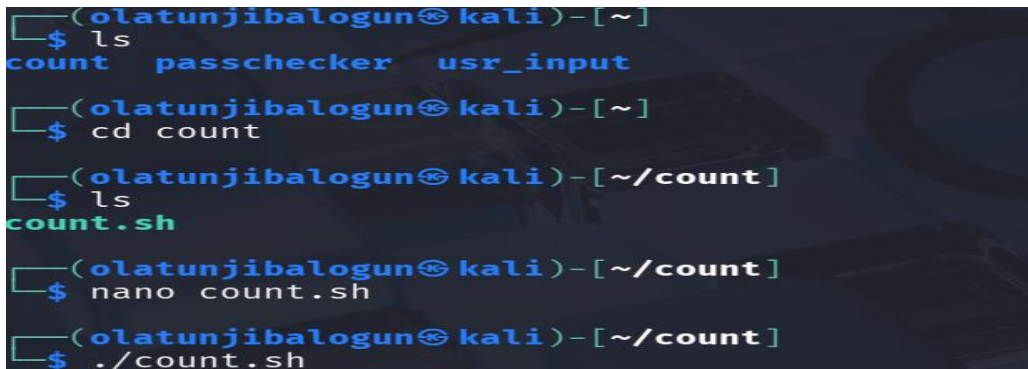
1 Script code

```
#!/bin/bash

# Countdown from 10 to 7
count=10
while [ $count -ge 7 ]
do
    echo "Countdown in progress: $count"
    count=$((count - 1))

done

# Final message
echo "Blast off!"
# work by
echo "By Olatunji Ramon Balogun"
```

A terminal window with a dark background and light blue text. The prompt is (olatumjibalogun@kali)-[~]. The user enters 'ls' and the output is 'count passchecker usr_input'. Then the user enters 'cd count' and the prompt changes to (olatumjibalogun@kali)-[~/count]. The user enters 'ls' and the output is 'count.sh'. Then the user enters 'nano count.sh' and the prompt changes to (olatumjibalogun@kali)-[~/count]. Finally, the user enters './count.sh' and the prompt changes to (olatumjibalogun@kali)-[~/count].

```
(olatumjibalogun@kali)-[~]
$ ls
count passchecker usr_input

(olatumjibalogun@kali)-[~]
$ cd count

(olatumjibalogun@kali)-[~/count]
$ ls
count.sh

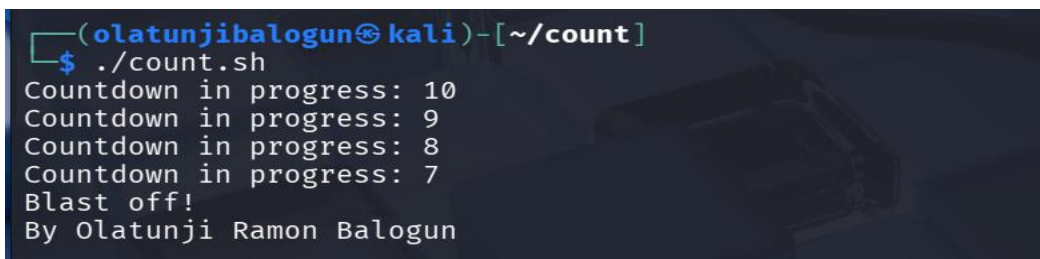
(olatumjibalogun@kali)-[~/count]
$ nano count.sh

(olatumjibalogun@kali)-[~/count]
$ ./count.sh
```

The Screenshot shows how I set up my countdown script by accessing the count directory and listing its contents (to verify that the script is there) and then using nano to edit the file and finally running it using ./count.sh.

A screenshot showing:

- Countdown in progress (minimum 3 numbers visible)
- Final "Blast off!" message
- Terminal showing it's your work

A terminal window with a dark background and light blue text. The prompt is (olatumjibalogun@kali)-[~/count]. The user enters './count.sh' and the output is 'Countdown in progress: 10', 'Countdown in progress: 9', 'Countdown in progress: 8', 'Countdown in progress: 7', 'Blast off!', and 'By Olatunji Ramon Balogun'.

```
(olatumjibalogun@kali)-[~/count]
$ ./count.sh
Countdown in progress: 10
Countdown in progress: 9
Countdown in progress: 8
Countdown in progress: 7
Blast off!
By Olatunji Ramon Balogun
```


The image illustrates that my countdown script has completed successfully. It counted down from ten to seven, printed "Countdown in Progress" before printing the final output "Blast Off!" and adding my name to the output.

```
(olatumjibalogun@kali)-[~/count]
$ cat count.sh
#!/bin/bash

# Countdown from 10 to 7
count=10
while [ $count -ge 7 ]
do
    echo "Countdown in progress: $count"
    count=$((count - 1))
done

# Final message
echo "Blast off!"
# work by
echo "By Olatunji Ramon Balogun"
```

The image provides a complete view of my countdown script, **count.sh**. This script uses a while statement to count down from ten to seven, print each number with a corresponding message, print "**Blast Off!**" after counting down, and include my name.

Exercise 5.1: Sum Function

Task: Create a function that takes two numbers and returns their sum.

- 2 Script code
- 3 A screenshot showing:
 - Function definition visible
 - At least two test cases with different numbers
 - Correct sum output

```
Session Actions Edit View Help
$ nano count.sh

(olatumjibalogun@kali)-[~/count]
$ cd ../

(olatumjibalogun@kali)-[~]
$ mkdir sum_function

(olatumjibalogun@kali)-[~]
$ cd sum_function/

(olatumjibalogun@kali)-[~/sum_function]
$ touch sum.sh

(olatumjibalogun@kali)-[~/sum_function]
$ nano sum.sh

(olatumjibalogun@kali)-[~/sum_function]
$ chmod +x sum.sh

(olatumjibalogun@kali)-[~/sum_function]
$ ls
sum.sh
```

Creating a Bash script environment — directories are created, a **sum.sh** file is added, made executable, and verified in Kali Linux.

```
(olatumjibalogun@kali)-[~/sum_function]
$ ./sum.sh
The sum of 10 7 is: 17
The sum of 2 8 is: 10
```

Running the Bash script (**sum.sh**) to calculate and display the sum of two numbers using a function.

```
(olatunjibalogun@kali)-[~/sum_function]
$ cat sum.sh
#!/bin/bash

# Function that calculate the sum
calculate_sum() {
    local num1=$1
    local num2=$2
    local sum=$((num1 + num2))
    echo "$sum"
}

# using the function
result=$(calculate_sum 10 7)
echo "The sum of 10 7 is: $result"    #correct sum output

result=$(calculate_sum 2 8)
echo "The sum of 2 8 is: $result"    #correct sum output
```

Viewing the contents of `sum.sh`, showing a Bash function that accepts parameters, performs arithmetic, and outputs the result.

Scripts code

```
#!/bin/bash

# Function that calculate the sum
calculate_sum() {
    local num1=$1
    local num2=$2
    local sum=$((num1 + num2))
    echo "$sum"
}

# using the function
result=$(calculate_sum 10 7)
echo "The sum of 10 7 is: $result"    #correct sum output

result=$(calculate_sum 2 8)
echo "The sum of 2 8 is: $result"    #correct sum output
```

Part 2: Practice Exercises (Choose any 2)

Select 2 exercises from the Easy, Medium, or Hard categories. For each chosen exercise, include the following in your PDF:

- 4 Clearly label which exercise you're completing.
- 5 Include your script code.
- 6 Embed screenshot(s) showing:
 - Code execution
 - Output/results
 - Your terminal/username

Easy Exercises

6. **Greeting Script:** Screenshot showing a time-based greeting.


```
Session Actions Edit View Help
olatumjibalogun@kali: ~/Greeting

(olatumjibalogun@kali)-[~]
$ cd Greeting/

(olatumjibalogun@kali)-[~/Greeting]
$ touch greeting.sh

(olatumjibalogun@kali)-[~/Greeting]
$ ls
greeting.sh

(olatumjibalogun@kali)-[~/Greeting]
$ ls -la
total 8
drwxrwxr-x  2 olatumjibalogun olatumjibalogun 4096 Dec 18 03:35 .
drwx----- 10 olatumjibalogun olatumjibalogun 4096 Dec 18 03:35 ..
-rw-rw-r--  1 olatumjibalogun olatumjibalogun   0 Dec 18 03:35 greeting.sh

(olatumjibalogun@kali)-[~/Greeting]
$ nano greeting.sh

(olatumjibalogun@kali)-[~/Greeting]
$ chmod +x greeting.sh

(olatumjibalogun@kali)-[~/Greeting]
$ ls -la
total 12
drwxrwxr-x  2 olatumjibalogun olatumjibalogun 4096 Dec 18 03:39 .
drwx----- 10 olatumjibalogun olatumjibalogun 4096 Dec 18 03:35 ..
-rwxrwxr-x  1 olatumjibalogun olatumjibalogun  196 Dec 18 03:39 greeting.sh
```

Creating and configuring the greeting script — a `greeting.sh` file is created, edited, and made executable in Kali Linux.

```
(olatumjibalogun@kali)-[~/Greeting]
$ ./greeting.sh
Hello, Olatunji
Good Morning

(olatumjibalogun@kali)-[~/Greeting]
$ cat greeting.sh
#!/bin/bash

# Greeting Script
# By Olatunji Ramon Balogun

echo "Hello, Olatunji"
hour=$(date +%H)

if [ "$hour" -lt 12 ]; then
    echo "Good Morning"
elif [ "$hour" -lt 18 ]; then
    echo "Good Afternoon"
else
    echo "Good Evening"
fi
```

Executing and reviewing the greeting script — the script displays a time-based greeting (morning, afternoon, or evening) based on the system hour.

Scripts code

```
#!/bin/bash

# Greeting Script
# By Olatunji Ramon Balogun

echo "Hello, Olatunji"
hour=$(date +%H)

if [ "$hour" -lt 12 ]; then
    echo "Good Morning"
elif [ "$hour" -lt 18 ]; then
    echo "Good Afternoon"
else
    echo "Good Evening"
fi
```

7. Number Guesser: Screenshot showing game interaction.

```
(olatumjibalogun@kali)-[~]
$ mkdir number_guesser

(olatumjibalogun@kali)-[~]
$ cd number_guesser/

(olatumjibalogun@kali)-[~/number_guesser]
$ touch number_guesser

(olatumjibalogun@kali)-[~/number_guesser]
$ nano number_guesser

(olatumjibalogun@kali)-[~/number_guesser]
$ ls -la
total 12
drwxrwxr-x  2 olatumjibalogun olatumjibalogun 4096 Dec 19 06:13 .
drwx----- 11 olatumjibalogun olatumjibalogun 4096 Dec 19 06:11 ..
-rw-rw-r--  1 olatumjibalogun olatumjibalogun  490 Dec 19 06:13 number_guesser

(olatumjibalogun@kali)-[~/number_guesser]
$ chmod +x number_guesser

(olatumjibalogun@kali)-[~/number_guesser]
$ ls -la
total 12
drwxrwxr-x  2 olatumjibalogun olatumjibalogun 4096 Dec 19 06:13 .
drwx----- 11 olatumjibalogun olatumjibalogun 4096 Dec 19 06:11 ..
-rwxrwxr-x  1 olatumjibalogun olatumjibalogun  490 Dec 19 06:13 number_guesser
```

Creating and configuring the number_guesser — a greeting.sh file is created, edited, and made executable in Kali Linux.

```
(olatumjibalogun@kali)-[~/number_guesser]
$ ./number_guesser
Hello, Olatunji
Welcome to the Number Guesser Game!
Guess a number between 1 and 10
Enter your guess: 7
Too high. Try again.
Enter your guess: 4
Too high. Try again.
Enter your guess: 1
Too low. Try again.
Enter your guess: 3
Correct! You guessed the number.
```

This script is a number guessing game where the correct number is set to 3. The user is asked to guess a number between 1 and 10. If the guess is lower than 3, the script displays “Too low”. If it is higher than 3, it displays “Too high”. When the user guesses 3, the script displays a success message and ends the game.

```
(olatumjibalogun@kali)-[~/number_guesser]
$ cat ./number_guesser
#!/bin/bash

# Number Guesser Game
# By Olatunji Ramon Balogun

echo "Hello, Olatunji"
echo "Welcome to the Number Guesser Game!"
echo "Guess a number between 1 and 10"

secret=3

while true; do
    read -p "Enter your guess: " guess

    if [ "$guess" -lt "$secret" ]; then
        echo "Too low. Try again."
    elif [ "$guess" -gt "$secret" ]; then
        echo "Too high. Try again."
    else
        echo "Correct! You guessed the number."
        break
    fi
done
```

This script is a guessing game where the user tries to guess a number.

- The correct number is set to 3 (secret=3)
- The user enters a guess using read -p
- The script checks the guess:
 - Too low → prints "Too low. Try again."
 - Too high → prints "Too high. Try again."
 - Correct → prints "Correct! You guessed the number." and ends the game

In short: The script compares the user's guess with the secret number and gives feedback until the correct number is guessed.

```
#!/bin/bash
```

Script code

```
# Number Guesser Game

# By Olatunji Ramon Balogun

echo "Hello, Olatunji"

echo "Welcome to the Number Guesser Game!"

echo "Guess a number between 1 and 10"

secret=3

while true; do

    read -p "Enter your guess: " guess

    if [ "$guess" -lt "$secret" ]; then

        echo "Too low. Try again."

    elif [ "$guess" -gt "$secret" ]; then

        echo "Too high. Try again."

    else

        echo "Correct! You guessed the number."

        break

    fi

done
```

8. File counter

```
(olatumjibalogun@kali)~[/file_counter]
$ ls
file_counter.sh

(olatumjibalogun@kali)~[/file_counter]
$ nano file_counter.sh

(olatumjibalogun@kali)~[/file_counter]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 19 08:07 .
drwx----- 12 olatumjibalogun olatumjibalogun 4096 Dec 19 08:04 ..
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 311 Dec 19 08:06 file_counter.sh

(olatumjibalogun@kali)~[/file_counter]
$ chmod +x file_counter.sh

(olatumjibalogun@kali)~[/file_counter]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 19 08:07 .
drwx----- 12 olatumjibalogun olatumjibalogun 4096 Dec 19 08:04 ..
-rwxrwxr-x 1 olatumjibalogun olatumjibalogun 311 Dec 19 08:06 file_counter.sh
```

Creating and configuring the file_counter — a greeting.sh file is created, edited, and made executable in Kali Linux.

```
(olatumjibalogun@kali)~[/file_counter]
$ ./file_counter.sh
Counting the number of files in your home directory!
You have 22 files in your home directory.

(olatumjibalogun@kali)~[/file_counter]
$ cat ./file_counter.sh
#!/bin/bash

# File Counter Script
# By Olatunji Ramon Balogun

echo "Counting the number of files in your home directory!"

# Count all regular files in the home directory
file_count=$(find "$HOME" -type f | wc -l)

echo "You have $file_count files in your home directory."
```

Shows the execution of a Bash script that counts and displays the total number of files in the user's home directory.

```
#!/bin/bash

# File Counter Script

# By Olatunji Ramon Balogun

echo "Counting the number of files in your home directory!"

# Count all regular files in the home directory

file_count=$(find "$HOME" -type f | wc -l)

echo "You have $file_count files in your home directory."
```

Medium Exercises

9. Password Generator: Screenshot showing different length passwords.

```
Session Actions Edit View Help
(olatumjibalogun@kali)-[~/Password_generator]
$ mkdir Password_generator
(olatumjibalogun@kali)-[~/Password_generator]
$ cd Password_generator/
(olatumjibalogun@kali)-[~/Password_generator]
$ touch password.sh
(olatumjibalogun@kali)-[~/Password_generator]
$ nano password.sh
(olatumjibalogun@kali)-[~/Password_generator]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 03:25 .
drwx----- 13 olatumjibalogun olatumjibalogun 4096 Dec 20 03:23 ..
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 229 Dec 20 03:24 password.sh
(olatumjibalogun@kali)-[~/Password_generator]
$ chmod +x password.sh
(olatumjibalogun@kali)-[~/Password_generator]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 03:25 .
drwx----- 13 olatumjibalogun olatumjibalogun 4096 Dec 20 03:23 ..
-rwxrwxr-x 1 olatumjibalogun olatumjibalogun 229 Dec 20 03:24 password.sh
```

Creating the Password Generator project directory, writing the Bash script, and setting executable permissions for password.sh.

```
(olatumjibalogun@kali)-[~/Password_generator]
$ ./password.sh
Hello, Olatunji
Password Generator
Enter desired password length (8-32): 15
Generated password: RZ1wL/TyIBGBiLh
(olatumjibalogun@kali)-[~/Password_generator]
$ cat password.sh
#!/bin/bash

# Password Generator
# By Olatunji Ramon Balogun

echo "Hello, Olatunji"
echo "Password Generator"

read -p "Enter desired password length (8-32): " length

# Generate password
password=$(openssl rand -base64 48 | cut -c1-"$length")
echo "Generated password: $password"
```

Executing the Password Generator script to generate a password of a specified length and displaying the script's source code for verification.

```
#!/bin/bash

# Password Generator
# By Olatunji Ramon Balogun

echo "Hello, Olatunji"
echo "Password Generator"

read -p "Enter desired password length (8-32): " length

# Generate password
password=$(openssl rand -base64 48 | cut -c1-"$length")
echo "Generated password: $password"
```


8 Website Status Checker: Screenshot showing the status of 3 websites.

```
(olatumjibalogun@kali)-[~]
$ cd Website_status/

(olatumjibalogun@kali)-[~/Website_status]
$ touch website_checker.sh

(olatumjibalogun@kali)-[~/Website_status]
$ nano website_checker.sh

(olatumjibalogun@kali)-[~/Website_status]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 04:00 .
drwx----- 14 olatumjibalogun olatumjibalogun 4096 Dec 20 03:57 ..
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 472 Dec 20 04:00 website_checker.sh

(olatumjibalogun@kali)-[~/Website_status]
$ chmod +x website_checker.sh

(olatumjibalogun@kali)-[~/Website_status]
$ ls -ls
total 4
4 -rwxrwxr-x 1 olatumjibalogun olatumjibalogun 472 Dec 20 04:00 website_checker.sh

(olatumjibalogun@kali)-[~/Website_status]
$ ls -la
total 12
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 04:00 .
drwx----- 14 olatumjibalogun olatumjibalogun 4096 Dec 20 03:57 ..
-rwxrwxr-x 1 olatumjibalogun olatumjibalogun 472 Dec 20 04:00 website_checker.sh
```

Creating and preparing the website status checker script — a `website_checker.sh` file is created, edited, listed, and assigned executable permissions in Kali Linux.

```
(olatumjibalogun@kali)-[~/Website_status]
$ ./website_checker.sh https://google.com
{"status":"up","code":301,"message":"https://google.com is reachable"}

(olatumjibalogun@kali)-[~/Website_status]
$ ./website_checker.sh https://ghostynoxethicalhacking.top
{"status":"down","code":000,"message":"https://ghostynoxethicalhacking.top is not reachable"}

(olatumjibalogun@kali)-[~/Website_status]
$ ./website_checker.sh https://github.com
{"status":"up","code":200,"message":"https://github.com is reachable"}
```

Executing the website status checker script — the script checks multiple URLs and returns JSON-formatted results indicating reachability and HTTP status codes.

```
(olatumjibalogun@kali)-[~/Website_status]
$ cat website_checker.sh
#!/bin/bash

# Website Status Checker
# By Olatunji Ramon Balogun

# Check if a website was provided
if [ -z "$1" ]; then
    echo '{"status":"error","message":"No website provided"}'
    exit 1
fi

site="$1"

# Add https:// only if missing
if [[ "$site" != http* ]]; then
    site="https://$site"
fi

# Get HTTP status code using curl
code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "$site")

# Check website status
if [[ "$code" -ge 200 && "$code" -lt 400 ]]; then
    echo "{\"status\": \"up\", \"code\": $code, \"message\": \"$site is reachable\"}"
else
    echo "{\"status\": \"down\", \"code\": $code, \"message\": \"$site is not reachable\"}"
fi
```


Viewing the website status checker script source code — the Bash script validates input, normalizes URLs, uses `curl` to retrieve HTTP status codes, and outputs structured status messages.

```
#!/bin/bash

# Website Status Checker
# By Olatunji Ramon Balogun

# Check if a website was provided
if [ -z "$1" ]; then
    echo '{"status":"error","message":"No website provided"}'
    exit 1
fi

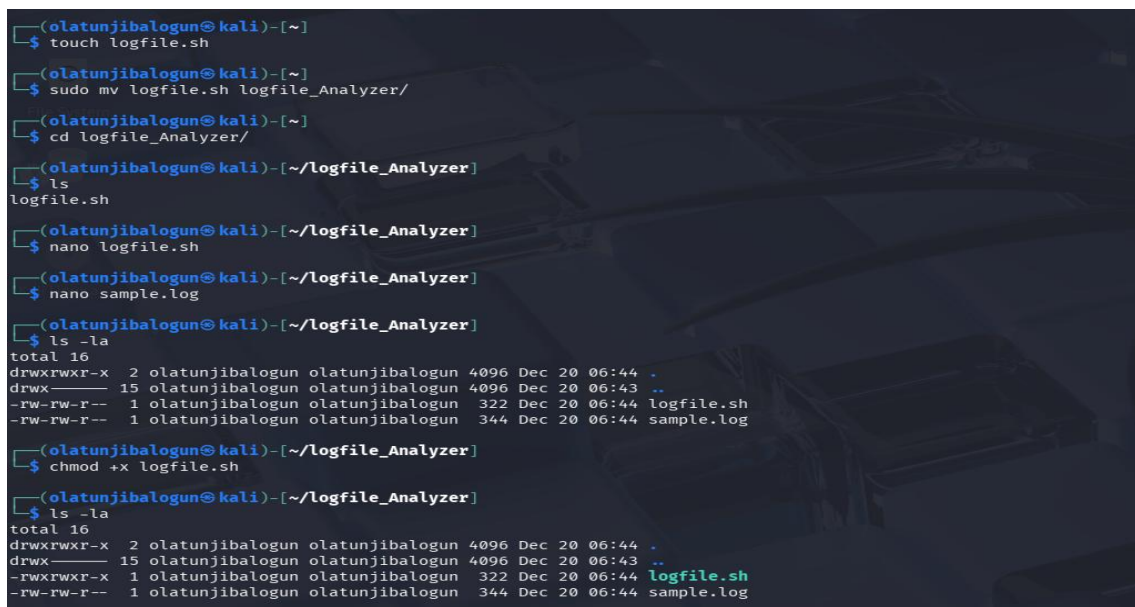
site="$1"

# Add https:// only if missing
if [[ "$site" != http* ]]; then
    site="https://$site"
fi

# Get HTTP status code using curl
code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "$site")

# Check website status
if [[ "$code" -ge 200 && "$code" -lt 400 ]]; then
    echo "{\"status\":\"up\", \"code\":$code, \"message\":\"$site is reachable\"}"
else
    echo "{\"status\":\"down\", \"code\":$code, \"message\":\"$site is not reachable\"}"
fi
```

7 Log File Analyzer: Screenshot showing the ERROR count (create a sample log).



```
(olatumjibalogun@kali)-[~]
└─$ touch logfile.sh
(olatumjibalogun@kali)-[~]
└─$ sudo mv logfile.sh logfile_Analyzer/
(olatumjibalogun@kali)-[~]
└─$ cd logfile_Analyzer/
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ ls
logfile.sh
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ nano logfile.sh
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ nano sample.log
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ ls -la
total 16
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 06:44 .
drwx----- 15 olatumjibalogun olatumjibalogun 4096 Dec 20 06:43 ..
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 322 Dec 20 06:44 logfile.sh
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 344 Dec 20 06:44 sample.log
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ chmod +x logfile.sh
(olatumjibalogun@kali)-[~/logfile_Analyzer]
└─$ ls -la
total 16
drwxrwxr-x 2 olatumjibalogun olatumjibalogun 4096 Dec 20 06:44 .
drwx----- 15 olatumjibalogun olatumjibalogun 4096 Dec 20 06:43 ..
-rwxrwxr-x 1 olatumjibalogun olatumjibalogun 322 Dec 20 06:44 logfile.sh
-rw-rw-r-- 1 olatumjibalogun olatumjibalogun 344 Dec 20 06:44 sample.log
```

Creating and setting up the Log File Analyzer script and sample log file, including file creation, editing, permission changes, and directory verification.

```
(olatumjibalogun@kali)~[/logfile_Analyzer]
$ ./logfile.sh sample.log
The log file 'sample.log' contains 3 ERROR(s).

(olatumjibalogun@kali)~[/logfile_Analyzer]
$
```

```
(olatumjibalogun@kali)~[/logfile_Analyzer]
$ cat logfile.sh
#!/bin/bash

# Log File Analyzer
# By Olatunji Ramon Balogun

# Check if log file exists
if [ ! -f "$1" ]; then
    echo "Usage: $0 <log_file>"
    exit 1
fi

logfile="$1"

# Count ERROR occurrences
error_count=$(grep -c "ERROR" "$logfile")

# Display result
echo "The log file '$logfile' contains $error_count ERROR(s)."
```

Running the Log File Analyzer script on the sample log file and displaying the total number of ERROR entries detected (3 errors found).

```
#!/bin/bash
echo "By: [OLatunji]"
# Create a sample log file
echo -e "INFO: System started\nERROR: Disk full\nINFO: User login\nERROR: Timeout" > sample.log

errors=$(grep -c "ERROR" sample.log)
echo "Number of ERROR entries: $errors"
```

Conclusion

The purpose of the Week 2 Lab was to give the students a basic understanding of how to write shell scripts in Linux. Students had hands-on experience on how to produce scripts related to user input, password checking, counting down, and performing basic arithmetic calculations (add, subtract, multiply, divide) during this week's Lab exercises.

The lab exercises also allowed students to work with processes that were not part of the curriculum, such as file management, system monitoring, and securing systems with password security.

In completing the Lab, students are now well-prepared to complete more challenging scripts in the upcoming weeks, thus beginning their journey into advanced Operating System concepts and technical tasks related to Administrative Processes